The Journal of Open Source Software

FAST-B: Fast Analytic B-spline Toolkit

Theodosios C. Theodosiou [1]

1 Dept. Energy Systems, University of Thessaly, Larissa, Greece.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 17 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

FAST-B: Fast Analytic B-spline Toolkit

Summary

FAST-B is a Python package that generates pseudo-analytic expressions for B-spline basis functions using NumPy instead of traditional symbolic computation libraries like SymPy. The library provides computationally efficient generation of B-spline expressions that are mathematically equivalent to those produced by symbolic approaches but with significantly improved performance. FAST-B enables rapid development of computational methods such as Isogeometric Analysis (IGA) by offering enhanced boundary condition support, memoization techniques, and efficient piecewise polynomial representations.

Statement of Need

B-splines are fundamental building blocks in computational geometry, computer-aided design (CAD), finite element analysis, and particularly in Isogeometric Analysis (IGA). Traditional approaches for generating analytic B-spline expressions rely on symbolic computation libraries like SymPy, which become computationally expensive for higher-order splines and complex knot vectors. The recursive nature of the Cox-de Boor algorithm (Cottrell, J. A., Hughes, T. J. R., Bazilevs, Y., 2009) combined with symbolic manipulation creates significant overhead that limits practical application in iterative design processes and real-time simulations.

Existing solutions face several limitations: - **Performance bottlenecks**: Symbolic computation becomes prohibitively slow for high-degree B-splines - **Memory inefficiency**: Repeated computations of identical B-spline expressions - **Limited boundary support**: Standard piecewise implementations restrict boundary condition flexibility - **Computational overhead**: Real-time applications require faster expression generation etc.

FAST-B addresses these limitations by leveraging NumPy's numerical efficiency while maintaining the analytic nature of the expressions. The library's innovative approach has already proven valuable in advanced applications, including the development of novel wavelet types (T.C. Theodosiou, 2021) where computational efficiency is crucial for practical implementation.

Key Features

1. Performance Optimization

- **NumPy-based Computation**: Replaces symbolic computation with efficient numerical operations while preserving analytic expression capabilities
- **Memoization System**: Implements automatic caching of computed B-spline functions and derivatives, achieving 10-1000× speedup for repeated access
- **Efficient Evaluation**: Optimized piecewise polynomial evaluation with minimal overhead

2. **Enhanced B-spline Support**

   - **Comprehensive B-spline Types**: Supports both cardinal (uniform) and non-uniform B-spline basis functions
   - **Recursive Implementation**: Implementation of the Cox-de Boor algorithm (Cottrell, J. A., Hughes, T. J. R., Bazilevs, Y., 2009) for mathematically correct results
   - **Arbitrary Degree Support**: Efficient handling of low to high-degree B-splines

3. **Advanced Boundary Condition Handling**

   - **Flexible Domain Support**: Beyond standard "open-open" or "closed-closed" domains, supports mixed "open-closed" and "closed-open" boundaries
   - **Piecewise Representation**: Maintains exact mathematical representation across domain boundaries
   - **Boundary-aware Operations**: All mathematical operations respect domain limitations and boundary conditions

4. **Mathematical Foundation**

   - **Polynomial Algebra**: Complete implementation of polynomial arithmetic with LaTeX representation capabilities
   - **Rational Functions**: Support for rational B-splines (NURBS) with efficient computation
   - **Automatic Differentiation**: Built-in derivative computation for B-spline basis functions

5. **Practical Applications**

   - **Isogeometric Analysis Ready**: Direct application to IGA formulations with demonstrated 1D rod problem implementation
   - **Wavelet Development**: Proven utility in developing derivative-orthogonal non-uniform B-spline wavelets (T.C. Theodosiou, 2021)
   - **Research and Education**: Suitable for academic research and teaching computational mathematics

Implementation

FAST-B is implemented in pure Python with NumPy as the primary computational backend. The architecture follows a hierarchical structure:

Core Components

1. **Polynomial Foundation** (`polynomial.py`):
   - Extends `numpy.poly1d` with enhanced operations
   - Provides LaTeX representation for mathematical documentation
   - Implements shifting, scaling, and differentiation operations
2. **Rational Function Support** (`rational.py`):
   - Represents ratios of polynomials for NURBS support
   - Maintains mathematical precision through exact operations
   - Supports all standard algebraic operations
3. **Piecewise Function Framework** (`function.py`, `branch.py`):
   - Manages domain-restricted function pieces
   - Handles boundary condition enforcement
   - Provides efficient evaluation across multiple domains
4. **B-spline Implementation** (`bspline.py`):
   - Implements Cox-de Boor recursive algorithm
   - Supports both cardinal and non-uniform variants
   - Includes automatic derivative computation
5. **Caching System** (`system.py`):

DRAFT

- Implements memoization pattern for performance
- Provides registry-based object management
- Ensures computational efficiency through intelligent caching

Performance Characteristics

The caching system demonstrates remarkable efficiency: - **First computation**: Full recursive Cox-de Boor algorithm execution - **Cached access**: O(1) retrieval with typical speedups of 10-1000× - **Memory efficiency**: Minimal overhead for cache storage - **Scalability**: Effective performance from low-degree to high-degree B-splines

Example Usage

Generation of a Cardinal (uniform) B-Spline

```
import numpy as np
from src.nurbs.bsplines import CardinalBSpline

# Create a quadratic B-spline basis function
B = CardinalBSpline(m=2)

# Compute first derivative (automatically cached)
B_prime = B.diff(1)

# Evaluate at multiple points
import numpy as np
x = np.linspace(0, 3, 1000)
y = B(x)
```

Generation of a Non-Uniform B-Spline

```
import numpy as np
from src.containers import Knotvector
from src.nurbs.bsplines import NonUniformBSpline

# Create non-uniform knot vector
t = Knotvector(degree=2, domain=(0, 1))
t.items = np.array([0, 0, 0, 0.3, 0.7, 1, 1, 1]) # Customize knotvector

# Create non-uniform B-spline basis function
N = NonUniformBSpline(t, 3, 1)  # degree=2, index=1
```

Availability

- **Operating System:** Cross-platform (Linux, macOS, Windows)
- **Programming Language:** Python 3.8+
- **Dependencies:** NumPy, Scipy, Matplotlib, Tabulate
- **License:** MIT License
- **Code Repository:** https://github.com/tctheodosiou/FAST-B
- **Documentation:** Comprehensive examples and API documentation
- **Support:** Issue tracking and community support via GitHub

DRAFT

Acknowledgements

The authors acknowledge the use of DeepSeek for proofreading docstrings and performance benchmarks. All core algorithmic implementations and mathematical formulations are human-authored.

References

Cottrell, J. A., Hughes, T. J. R., Bazilevs, Y. (2009). *Isogeometric Analysis: Toward Integration of CAD and FEA*. ISBN: 978-0-470-74873-2

T.C. Theodosiou. (2021). Derivative-orthogonal non-uniform B-Spline wavelets. *Mathematics and Computers in Simulation*, *188*, 368–388. https://doi.org/10.1016/j.matcom.2021.04.012

DRAFT